

Testing Policy

Overview

This document defines testing standards for the VirtualController project. All components must pass corresponding test suites before merge. Testing spans three levels: unit tests (core logic), functional/integration tests (component interactions), and end-to-end tests (full workflows).

Unit Tests (39 tests total)

Pure logic tests with zero external dependencies. These validate business logic in isolation using mock data structures.

Server & Game Logic (10 tests)

File: `tests/server/test_game_logic.cpp`

Scope: Core game state management independent of networking

Coverage:

- Player initialization and count boundaries
- Game state transitions (LOBBY → ACTIVE → ENDED)
- Input application and collision logic
- Alive player tracking

Player ID Assignment (7 tests)

File: `tests/server/test_player_ids.cpp`

Scope: Multi-PC player ID mapping logic

Coverage:

- Sequential ID assignment for multiple controllers per PC
- Boundary conditions (0, 1, 4 controllers)
- Multi-connection scenarios with offset calculations

Controller Management (6 tests)

File: `tests/launcher/test_server_sync.cpp`

Scope: Launcher's controller hotplug detection

Coverage:

- First connection updates
- Duplicate suppression
- Change detection on controller add/remove
- Count tracking over time

Snake Game Mechanics (10 tests)

File: `tests/snake/test_game_mechanics.cpp`

Scope: Movement, collision, and boundary logic

Coverage:

- Movement in all four directions
- Wall collision detection
- Self-collision and body tracking
- Input validation (opposite directions)

Karting Game Mechanics (6 tests)

File: `tests/karting/test_karting_mechanics.cpp`

Scope: Physics, lap tracking, and display logic

Coverage:

- Speed clamping (acceleration, reverse, friction)
- Steering and rotation response
- Lap completion thresholds
- End-screen lap display formatting

Run: `ctest -C Release` or directly: `build/tests/Release/test_*.exe`

Pass Criteria: 100% pass rate; any failure blocks PR

Functional/Integration Tests (51+ tests planned)

Tests validating component interactions with real network communication and shared state.

Networking Tests (24 tests)

File: `tests/test_networking.py`

Scope: Server ↔ Client JSON protocol and state synchronization

Coverage:

- CONNECT message parsing (controller count updates)
- INPUT message delivery and player ID mapping
- STATE_UPDATE broadcast to all clients
- Connection lifecycle (join/leave/reconnect)
- Concurrent multi-client scenarios

CI Build: Yes (depends on C++ build)

Run: `python tests/test_networking.py -v`

Protocol Tests (27 tests)

File: `tests/test_protocol.py`

Scope: JSON message format validation

Coverage:

- Message type parsing (CONNECT, INPUT, START, RESET)
- Field presence and type validation
- Multi-line message buffering
- Error handling for malformed JSON

CI Build: Yes

Run: `python tests/test_protocol.py -v`

Pass Criteria: 100% pass rate; failures indicate protocol incompatibility

End-to-End Tests

Full game workflows from launcher startup through gameplay completion.

Snake Game Flow

Scope: Launcher → Server → Snake Client → Gameplay → Results

Coverage:

- Controller detection on launcher startup
- Game start flow
- Multi-player input handling
- Collision, food consumption, and scoring
- Game end and reset

Karting Game Flow

Scope: Launcher → Server → Karting Client → Race → Results

Coverage:

- Track spawning and collision detection
 - 4-player simultaneous racing
 - Lap counting and finish detection
 - Results display with player rankings
-

Running Tests Locally

```
# Configure with tests enabled
cd build
cmake .. -DQt6_DIR="<path>" -DSFML_DIR="<path>" -DBUILD_TESTS=ON

# Build all tests
cmake --build . --config Release --target test_game_logic
```

```
test_player_ids test_controller_management test_game_mechanics  
test_karting_mechanics
```

```
# Run tests
```

```
cd tests/Release
```

```
./test_game_logic.exe
```

```
./test_karting_mechanics.exe
```

```
# ... etc
```

```
# Or run all via CTest (requires proper test registration in  
CMakeLists.txt)
```

```
cd ../..
```

```
ctest -C Release --output-on-failure
```